# RB2301 CA1 Report

| No. | Name | Student ID |
|---|---|---|
| 1. | Maxwell Timothy Ebora Guico | A0323869N |
| 2. | Suda Jesmitha Reddy | A0321806L |
| 3. | Nguyen Linh Nhien | A0325663B |

## Introduction

The CA1 problem is to move a mobile robot through a field of coke cans without using a pre-built map. The robot receives a two-dimensional LiDAR scan and publishes velocity commands through ROS 2. It must decide from the current scan whether the path ahead is clear, whether a visible gap is wide enough, or whether it should move toward a safer side.

This problem is harder than simply looking for the largest distance reading. The LiDAR measures from one point on the robot, but the robot has a real width. Two cans can appear far apart when their separation is measured diagonally, while the space across the robot's planned route is still too narrow. A can that is closer to the robot can also block the route to a gap that looks open farther away.

## Overview of the node

ObstacleAvoidanceNode is the main ROS 2 node for obstacle avoidance. It subscribes to /scan for LiDAR data, publishes motion commands to /cmdvel, and runs the selected algorithm through a 20 Hz timer callback. The node's algo parameter selects either "nhien" or "max" (e.g 'ros2 launch rb2301_ca1 ca1.launch.py algo:=max random_seed:=5 scenario:=random'). The node stores every second LiDAR range reading in a NumPy array, while move_2D(x, y, turn) clips and publishes forward or backward velocity (x, ±0.4 m/s), lateral velocity (y, ±0.4 m/s), and rotational velocity (turn, ±0.8 rad/s).

## Methodology

### Checking the Forward Corridor

The LiDAR marks the visible surface of each can with 2 degrees between each point from the robot. For the straight-ahead check, the controller uses a rectangular strip that begins at the robot, extends 0.6 m forward, and reaches 0.12 m to each side of an imaginary line through the middle of the robot. The strip is therefore 0.24 m wide. If any can return lies inside it, straight motion is blocked. Otherwise, the robot moves forward at 0.30 m/s. After avoiding a can, it requires eight clear scans (0.40 s) before returning to straight motion.

### Grouping Cans and Finding Gaps

Consecutive LiDAR returns are treated as the same can when their detected locations are no more than 0.12 m apart. For two neighboring cans, the controller uses the right inner edge of one can and the left inner edge of the next can. The gap midpoint is exactly halfway between these two edges.

The controller only considers gap midpoints inside the 170-degree cone directly in front of the robot: from 85 degrees left to 85 degrees right. A gap must also be at least 0.33 m wide across the planned travel direction and must not have a nearer can block the route.

### Choosing a Motion

The robot first chooses a safe gap that is almost straight ahead. If none exists, the current code checks right-side gaps first because its preferred-side setting is "right". The robot then moves at 0.18 m/s toward the gap midpoint, which is halfway between the two inner cans' edges. If no suitable right-side gap exists, the fallback compares the measured space on both sides.

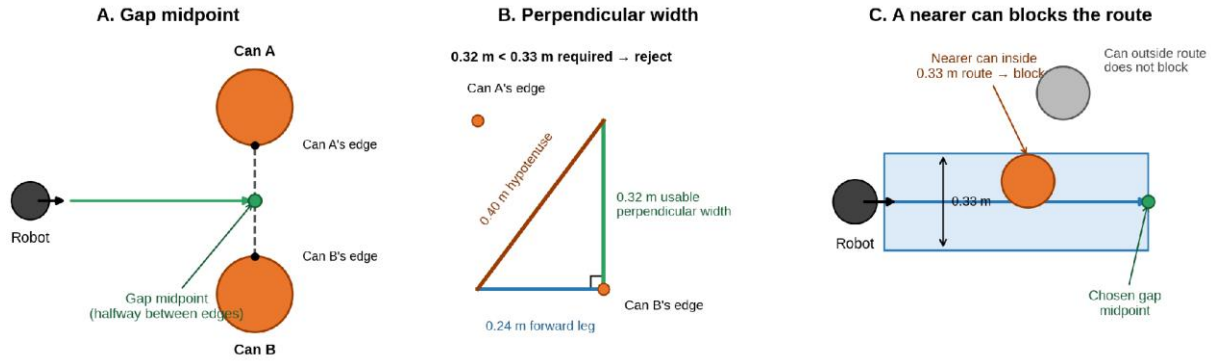
### Fallback and Close-Range Safety

If no safe opening remains, the controller compares the median LiDAR distance on each side and selects the clearer one. Before moving sideways at 0.25 m/s, it checks a 0.30 m path beside the robot. If that side is blocked and the opposite side is open, it switches and holds the new direction for 12 cycles. When both sides are blocked or the front is within 0.20 m, the robot reverses at 0.08 m/s.

### The Math Behind the Algorithm

Each LiDAR reading gives a distance r and an angle theta. The controller uses these values to calculate the detected location relative to the robot. Positive x means forward and positive y means left.

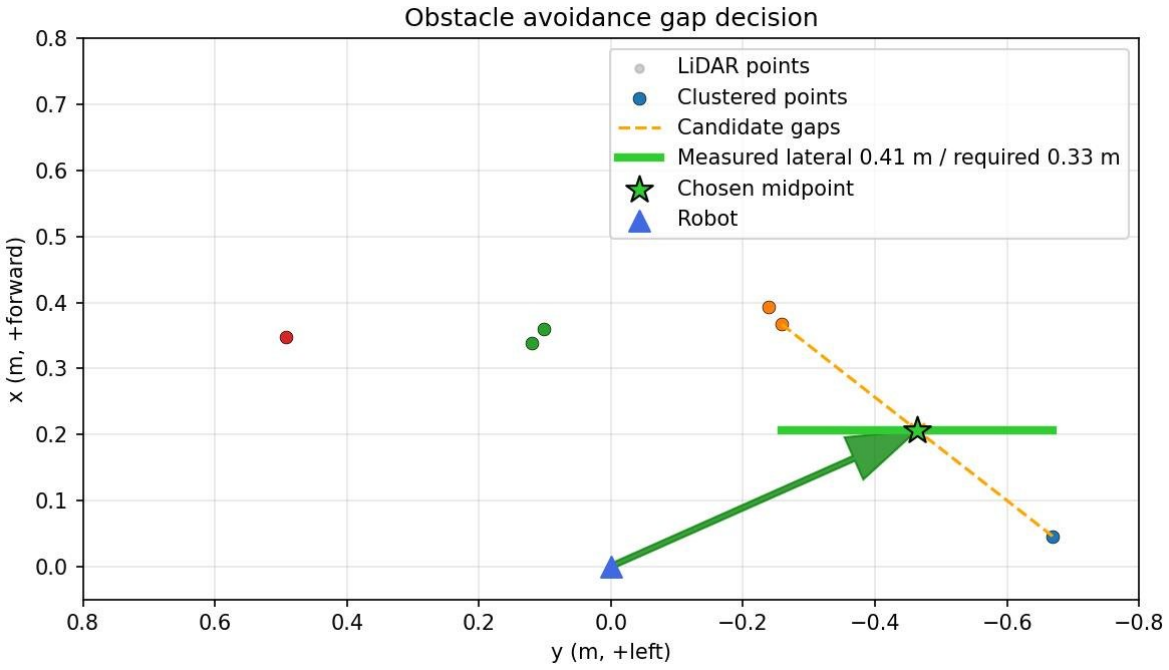
$$x = r\cos(\theta), \qquad y = r\sin(\theta)$$




### Perpendicular Gap Width

Let g be the line between the two inner can edges. Let d be a unit vector that points from the robot to the gap midpoint. The useful gap is the part of g that is perpendicular to d. In two dimensions, the controller calculates this width using the following cross-product expression.

$$w_{\perp} = |d_x g_y - d_y g_x|$$



As shown in the figure, the robot evaluates the candidate gap marked by the orange dashed line and projects it perpendicular to the planned travel direction. This produces a usable lateral width of 0.41 m, shown by the green line. Because this exceeds the required clearance of 0.33 m, the gap is accepted, and its midpoint marked by the green star, is selected as the robot's next target.

**Robot Width and Route Check**

$$w_{\min} = 0.23 + 2(0.05) = 0.33 \text{ m}$$

The robot is 0.23 m wide and the controller adds 0.05 m of safety space on each side, giving a required width of 0.33 m. The controller then checks the whole route from the robot to the chosen gap midpoint.

## Code Explanation

| Function | Explanation |
|---|---|
| timer_callback() | Repeatedly checks the latest LiDAR data, decides whether the path ahead is clear, chooses a safe gap or sideways escape, and sends a movement command. |
| find_gap_targets() | Groups LiDAR points that belong to the same obstacle, finds openings between obstacles, and keeps openings that are wide and clear enough for the robot. |
| choose_gap() | Chooses the best usable opening. It prefers a gap directly ahead; otherwise, it picks one on the preferred side that needs the least sideways movement. |
| get_sidestep_command() | Provides an emergency movement when no suitable gap is found. It moves toward the clearer side, changes direction if blocked, and reverses slightly if an obstacle is extremely close ahead. |
| get_front_scan() | Takes the front half of the LiDAR scan, removes invalid readings, and converts distances into forward and sideways coordinates. |
| required_clearance_for_direction() | Calculates the empty space needed to travel in each direction, accounting for the robot's size and safety margin. |
| move_2D() | Sends the movement command, controlling forward, sideways, and turning speeds within safe limits. |
| sub_scan_callback() | Receives new LiDAR measurements and stores a reduced scan for the main controller. |
| __init__() | Sets up the ROS node, including publishers, subscribers, timers, movement settings, safety distances, evaluation variables, and debugging options. |

## Empirical Results

The figure below are the simulated run times in seconds, path length taken by the robot in meters, the number of cans present and whether the run was successful. Height, width, edge case spacing and size_div were kept constant with the seed value being the only variable changed. The following are the parameters in the obstacle_generator.py

| **Parameters** | **height** | **size_div** | **width** | **randomise** | **Edge_case_spacing** |
|---|---|---|---|---|---|
| **Value** | 20 | 5 | 22 | True | 0.2 |

| Seed | Time (simulated s) | Path length (m) | Success |
|---|---|---|---|
| 1 | 26.28 | 7.220 | Yes |

| | | | |
|---|---|---|---|
| 2 | 26.46 | 7.281 | Yes |
| 3 | 24.94 | 7.161 | Yes |
| 4 | 27.48 | 7.420 | Yes |
| 5 | 24.84 | 7.162 | Yes |
| **Total** | **130.00** | **36.244** | **5/5** |
| **Average** | **26.00** | **7.249** | — |

| Parameter | Number of runs | Success Rate (%) | Average Time Taken (s) | Average Path Length (m) |
|---|---|---|---|---|
| algo = nhien | 5 | 100 | 19.86 | 7.5598 |
| algo = max | 5 | 100 | 26.00 | 7.249 |

Comparing the two algorithms, algo = nhien achieved a lower average completion time of 19.86 s, compared to 26.00 s for algo = max. However, algo = max produced a slightly shorter average path length of 7.249 m, compared to 7.5598 m for algo = nhien. Both algorithms achieved a 100% success rate across five runs. The difference in completion time may be partly attributed to the different movement speeds used by the two algorithms, while the difference in path length reflects variations in their obstacle-avoidance and path-selection strategies.

## Reflections

This CA1 has opened our eyes to how simple problems that humans take for granted, like navigating obstacles can be such a difficult challenge for a robot. The idea we had for the robot to navigate through the course was deceivingly hard. At first, we came up thinking about the gaps between the coke cans as vectors, and we simply want to go to midpoint. However, there are many considerations we did not think of. One of which was the fact that just because the vector length between the two cans is larger than the robot, it doesn't mean that the robot can fit through. As we drew the problem out, we noticed that we should have been taking the perpendicular distance to ensure that the robot's width, together with a safety radius, can pass through. There were many edges cases we had to account for such as L shaped arrangement of cans, diagonal cans setup that we have put in the video, which drastically changed how our robot functions. Furthermore, the problem of having many possible vectors between two cans because the LIDAR points were not initially clustered came up.

There were some sim-to-real gaps, such as how in simulation it looks like its strafing and not moving diagonally but in real life demonstration, it works as intended. All in all, just from a simple problem, we had to think fundamentally how the robot "sees" and "thinks" to be able to debug and improve our algorithm. This also gave us a greater appreciation of other algorithms such as SLAM, that are more robust in solving these issues.

Github Repo: Source Code

## AI Declaration

I used GPT-6 Sol to generate code and visualisations for my gap following algorithm. I am responsible for the content and quality of the submitted work.